Scene Audio Manager

Installation and Usage Manual

1. Overview

Scene Audio Manager is a lightweight Unity audio helper system designed to play sound effects from Unity Events and Animation Events without manually assigning audio clips on every animated object.

The system uses one central audio manager in the scene. Each sound is registered as an audio entry with an ID, one or more audio clips, volume, pitch and optional random pitch settings. Animated objects use an `AnimationAudioRelay` component to request sounds from the scene manager.

This setup is useful for footsteps, attacks, impacts, UI sounds, object interactions and any animation-driven sound effect.

---

2. Included Scripts

The system contains these files:

Runtime Scripts

SceneAudioManager.cs
The main scene-level audio manager. It stores all audio entries and plays sounds by ID or index.

AnimationAudioRelay.cs
A small relay component placed on animated GameObjects. It receives Animation Events and forwards the requested sound ID to the active `SceneAudioManager`.

Editor Script

SceneAudioManagerEditor.cs
A custom Unity inspector for `SceneAudioManager`. It displays the audio entries and adds Play and Stop buttons directly inside each audio entry row.

Setup Note

AudioManagerAnimationEvent.txt
A simple reminder for the correct Animation Event setup:

Function: `PlaySound`
String field: the sound ID

---

3. Requirements

Recommended Unity version:

Unity 2021 LTS or newer.

The current scripts include compatibility handling for newer Unity versions by using `FindAnyObjectByType` when available and falling back to `FindObjectOfType` on older versions.

The system requires:

- Unity Audio Clips imported into the project
- One `SceneAudioManager` in the scene
- `AnimationAudioRelay` on any animated object that needs to play sounds from Animation Events
- An `AudioSource` on the same GameObject as the relay, added automatically by the script requirement

---

4. Installation

Step 1: Create folders

Inside your Unity project, create this folder structure:

```
Assets/
  AudioManager/
    Runtime/
    Editor/
```

Step 2: Add the runtime scripts

Place these files inside:

```
Assets/AudioManager/Runtime/
```

Files:

```
SceneAudioManager.cs
AnimationAudioRelay.cs
```

Step 3: Add the editor script

Place this file inside:

```
Assets/AudioManager/Editor/
```

File:

```
SceneAudioManagerEditor.cs
```

Important:
`SceneAudioManagerEditor.cs` must stay inside an `Editor` folder because it uses `UnityEditor`. If it is placed in a normal runtime folder, Unity builds can fail.

Step 4: Let Unity compile

After adding the files, return to Unity and wait for the scripts to compile.

If there are no compile errors, the system is ready to set up in a scene.

---

5. Scene Setup

Step 1: Create the manager object

In the Hierarchy:

1. Right-click in the scene.
2. Select `Create Empty`.
3. Rename it to:

```
Scene Audio Manager
```

Step 2: Add SceneAudioManager

With the object selected:

1. Click `Add Component`.
2. Search for:

```
SceneAudioManager
```

1. Add it.

The script requires an `AudioSource`, so Unity will automatically add an `AudioSource` component if one is missing.

The manager automatically sets the Audio Source defaults:

```
Play On Awake: Disabled
Volume: 1
Pitch: 1
```

---

6. Creating Audio Entries

In the `SceneAudioManager` inspector, expand `Audio Entries`.

Each entry represents one playable sound definition.

Audio Entry Fields

Field	Description
ID	The name used to call this sound from Unity Events or Animation Events.
Clips	A list of one or more Audio Clips.
Volume	Playback volume from 0 to 1. Default is 1.
Pitch	Fixed pitch value from 0.1 to 3. Default is 1.
Use Random Pitch	Enables random pitch variation.
Random Pitch Min	Minimum pitch when random pitch is enabled.
Random Pitch Max	Maximum pitch when random pitch is enabled.

Example entries

```
ID: Footstep
Clips:
  footstep_grass_01
  footstep_grass_02
  footstep_grass_03
Volume: 0.8
```

```
Pitch: 1
Use Random Pitch: Enabled
Random Pitch Min: 0.9
Random Pitch Max: 1.1
```

```
ID: SwordSwing
Clips:
  sword_swing_01
  sword_swing_02
Volume: 1
Pitch: 1
Use Random Pitch: Enabled
Random Pitch Min: 0.95
Random Pitch Max: 1.05
```

```
ID: DoorOpen
Clips:
  door_open
Volume: 1
Pitch: 1
Use Random Pitch: Disabled
```

---

7. How Clip Selection Works

If an audio entry contains multiple clips, the manager randomly selects one valid clip when the sound is played.

Null clip slots are ignored.

This means you can safely add several variations to the same ID, such as multiple footstep, hit or swing sounds.

Example:

```
ID: Footstep
Clips:
  footstep_01
  footstep_02
  footstep_03
```

Calling `Footstep` will randomly play one of the assigned clips.

---

8. Playing Sounds from Unity Events

You can call sounds directly from Unity Events using the `SceneAudioManager`.

Step 1: Add the target object

In the Unity Event field:

1. Drag the `Scene Audio Manager` object into the event target slot.
2. Open the function dropdown.
3. Select:

```
SceneAudioManager > PlayByName(string)
```

Step 2: Enter the sound ID

In the string field, enter the audio entry ID.

Example:

```
DoorOpen
```

When the Unity Event is triggered, the manager plays the audio entry with that ID.

---

9. Playing Sounds from Animation Events

Animation Events are the recommended workflow for syncing sounds with animation frames.

Use this for footsteps, attacks, weapon swings, land impacts and other sounds that must happen at an exact animation time.

Step 1: Add AnimationAudioRelay to the animated object

Select the animated GameObject that owns the Animator and receives Animation Events.

Add this component:

```
AnimationAudioRelay
```

The relay automatically requires and uses an `AudioSource` on the same object.

Step 2: Make sure the scene has a SceneAudioManager

The relay automatically searches for the active `SceneAudioManager` at runtime.

You do not need to drag the manager manually into the relay.

Step 3: Open the animation clip

1. Select the animated object.
2. Open the `Animation` window.
3. Choose the animation clip.
4. Move the timeline to the exact frame where the sound should play.

Step 4: Add the Animation Event

Add a new Animation Event at the chosen frame.

Set the event like this:

```
Function: PlaySound
String: Footstep
```

Replace `Footstep` with the ID from your `SceneAudioManager`.

Example:

```
Function: PlaySound
String: SwordSwing
```

When the animation reaches that frame, Unity calls `PlaySound`, the relay forwards the ID to the manager and the sound plays through the local Audio Source.

---

10. Playing Sounds by Index

The system also supports playing sounds by index.

The index is based on the order of entries in the `Audio Entries` list.

Example:

```
PlaySoundByIndex(0)
```

This plays the first audio entry.

Using IDs is usually safer than using indexes because IDs do not break if the list order changes.

Recommended workflow:

```
Use PlaySound(string soundId) for Animation Events.
Use PlayByName(string clipId) for Unity Events.
```

---

11. Previewing Sounds in the Inspector

The custom inspector adds Play and Stop buttons directly inside each audio entry row.

Edit Mode Preview

When Unity is not in Play Mode:

- Press `Play` on an audio entry to preview one clip from the entry.
- Press `Stop` to stop preview playback.
- The preview uses Unity editor preview playback.

Important:
Exact runtime volume, pitch and random pitch behavior is only fully reliable in Play Mode.

Play Mode Preview

When Unity is in Play Mode:

- Press `Play` on an audio entry.
- The manager plays the entry using its runtime settings.
- Volume, pitch and random pitch are applied by the manager.

Use Play Mode when testing the final in-game sound behavior.

---

12. Recommended Workflow

For footsteps

1. Create one audio entry called:

```
Footstep
```

1. Add several footstep clips.
2. Enable random pitch.
3. Use a small range:

```
Random Pitch Min: 0.9
Random Pitch Max: 1.1
```

1. Add Animation Events on each footstep frame:

```
Function: PlaySound
String: Footstep
```

For attacks

1. Create an entry:

```
SwordSwing
```

1. Add one or more swing clips.
2. Add the Animation Event on the frame where the weapon starts moving quickly:

```
Function: PlaySound
String: SwordSwing
```

For impacts

1. Create an entry:

```
Impact
```

1. Add one or more impact clips.
2. Trigger it from an Animation Event, collision event or Unity Event depending on your system.

---

13. Runtime API

SceneAudioManager

Play a sound by ID:

```
sceneAudioManager.PlayByName("Footstep");
```

Play a sound by ID through a specific Audio Source:

```
sceneAudioManager.PlayByName("Footstep", targetAudioSource);
```

Play a sound by list index:

```
sceneAudioManager.PlayByIndex(0);
```

Stop the manager Audio Source:

```
sceneAudioManager.StopAudio();
```

Stop a specific Audio Source:

```
sceneAudioManager.StopAudio(targetAudioSource);
```

Set manager Audio Source volume:

```
sceneAudioManager.SetVolume(1f);
```

Set manager Audio Source pitch:

```
sceneAudioManager.SetPitch(1f);
```

AnimationAudioRelay

Play sound from Animation Event:

```
public void PlaySound(string soundId)
```

Play sound by index from Animation Event:

```
public void PlaySoundByIndex(int index)
```

Stop the local Audio Source:

```
public void StopLocalAudio()
```

---

14. Troubleshooting

Sound does not play from Animation Event

Check the following:

1. The animated object has `AnimationAudioRelay`.
2. The scene has one active `SceneAudioManager`.
3. The Animation Event function is set to:

```
PlaySound
```

1. The String field contains a valid audio ID.
2. The ID in the Animation Event matches the ID in `SceneAudioManager`.
3. The audio entry has at least one assigned Audio Clip.
4. The Audio Listener exists in the scene, usually on the Main Camera.
5. The object is active when the animation event is triggered.

Console warning: No SceneAudioManager found in the scene

This means the relay could not find the manager.

Fix:

1. Create an empty GameObject.
2. Add `SceneAudioManager`.
3. Make sure the object is active in the scene.

Console warning: No audio clip found with id

This means the sound ID does not exist in the manager lookup.

Fix:

1. Open the `SceneAudioManager`.
2. Check the `Audio Entries` list.
3. Make sure the ID is not empty.
4. Make sure the Animation Event string uses the same ID.

Example:

Manager ID:

```
Footstep
```

Animation Event String:

```
Footstep
```

Console warning: Audio entry has no clips assigned

The ID exists, but there are no valid clips in the Clips list.

Fix:

1. Expand the audio entry.
2. Add at least one Audio Clip.
3. Remove empty clip slots if needed.

Play button in the inspector does not preview correctly

Check the mode:

- In Edit Mode, Unity uses editor preview playback.
- In Play Mode, the manager uses runtime playback.

For accurate volume, pitch and random pitch testing, enter Play Mode and use the Play button again.

Random pitch sounds wrong

Check:

```
Random Pitch Min
Random Pitch Max
```

Recommended subtle ranges:

```
0.95 to 1.05
0.9 to 1.1
```

Avoid extreme values unless you want stylized results.

Multiple sounds overlap

This is expected because the system uses one-shot playback.

One-shot playback allows a sound to play without automatically stopping previously triggered sounds.

This is usually good for footsteps, impacts and combat sounds.

---

15. Best Practices

Use clear IDs

Recommended:

```
Footstep_Grass
Footstep_Stone
SwordSwing
SwordImpact
DoorOpen
DoorClose
```

Avoid vague IDs:

```
Sound1
Audio2
Test
```

Prefer ID playback over index playback

IDs are easier to maintain.

Indexes can break if you reorder the list.

Use multiple clips for repeated sounds

Repeated sounds such as footsteps should use multiple clips and random pitch to avoid sounding robotic.

Keep random pitch subtle

For natural variation, use small pitch ranges.

Recommended:

```
0.95 to 1.05
```

For stronger variation:

```
0.9 to 1.1
```

Keep editor scripts inside an Editor folder

Any script using `UnityEditor` must stay inside an `Editor` folder.

Recommended location:

```
Assets/AudioManager/Editor/SceneAudioManagerEditor.cs
```

---

16. Quick Setup Checklist

Use this checklist when adding the system to a new scene:

```
[ ] SceneAudioManager.cs is in a runtime folder.
[ ] AnimationAudioRelay.cs is in a runtime folder.
[ ] SceneAudioManagerEditor.cs is inside an Editor folder.
[ ] Scene has a GameObject with SceneAudioManager.
[ ] Audio entries have valid IDs.
[ ] Audio entries have at least one Audio Clip.
[ ] Animated object has AnimationAudioRelay.
[ ] Animation Event function is PlaySound.
[ ] Animation Event String field contains the correct sound ID.
[ ] Scene has an Audio Listener.
[ ] Tested preview in Play Mode for exact pitch and volume behavior.
```

---

17. Example Animation Event Setup

For a footstep sound:

SceneAudioManager entry:

```
ID: Footstep
Clips:
  footstep_01
  footstep_02
  footstep_03
Volume: 0.8
Pitch: 1
Use Random Pitch: Enabled
Random Pitch Min: 0.95
Random Pitch Max: 1.05
```

Animation Event:

```
Function: PlaySound
String: Footstep
```

For a sword swing:

SceneAudioManager entry:

```
ID: SwordSwing
Clips:
  sword_swing_01
  sword_swing_02
Volume: 1
Pitch: 1
Use Random Pitch: Enabled
Random Pitch Min: 0.95
Random Pitch Max: 1.05
```

Animation Event:

```
Function: PlaySound
String: SwordSwing
```

---

18. Final Notes

The Scene Audio Manager is designed to keep audio setup centralized while still allowing each animated object to play sounds through its own local Audio Source.

This keeps Animation Events simple:

```
PlaySound + Sound ID
```

The manager handles clip selection, volume, pitch and random pitch automatically.